

METHOD OVERLOADING AND METHOD OVERRIDING

Aim:

To understand and implement method overloading and method overriding.

PRE LAB EXERCISE

QUESTIONS

- ✓ What is method overloading?
 - Method overloading means having multiple methods with the same name in the same class.
 - The methods must differ in number, type, or order of parameters.
 - It is an example of compile-time polymorphism in Java.
 - It improves readability and flexibility by allowing similar operations with different inputs.

- ✓ What is method overriding?
 - Method overriding occurs when a subclass provides a new implementation for a method already defined in its parent class.
 - The method name and parameter list must be exactly the same as in the parent class.
 - It is an example of run-time polymorphism in Java.
 - It allows subclasses to provide specific behavior based on their requirements.

- ✓ Difference between overloading and overriding.

Method Overloading	Method Overriding
Same method name with different parameters.	Same method name with same parameters.
Occurs within the same class.	Occurs between parent and child classes.
Resolved at compile time.	Resolved at run time.
Does not require inheritance.	Requires inheritance.

IN LAB EXERCISE

Objective:

To demonstrate compile-time and runtime polymorphism.

PROGRAMS:

1.Student Result System (Method Overriding)

Description:

- Base class Student has method displayResult().
- Subclasses UGStudent and PGStudent override the method to show different grading systems.

Code :

```
import java.util.Scanner;
```

```
// Base class
```

```
class Student {
```

```
    String name;
```

```
    void displayResult() {
```

```
        System.out.println("Student Result");
```

```
    }
```

```
}
```

```
// UG Student subclass
```

```
class UGStudent extends Student {
```

```
    int marks;
```

```
    UGStudent(String n, int m) {
```

```
        name = n;
```

```
        marks = m;
```

```
    }
```

```
@Override
```

```
void displayResult() {  
    double percentage = (marks / 100.0) * 100;  
    System.out.println("UG Student: " + name);  
    System.out.println("Marks: " + marks);  
    System.out.println("Percentage: " + percentage + "%");  
}  
}
```

// PG Student subclass

```
class PGStudent extends Student {  
    double gpa;  
  
    PGStudent(String n, double g) {  
        name = n;  
        gpa = g;  
    }  
}
```

@Override

```
void displayResult() {  
    System.out.println("PG Student: " + name);  
    System.out.println("GPA: " + gpa + " / 10");  
}  
}
```

// Main class

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        // Input for UG student  
        System.out.print("Enter UG Student Name: ");
```

```

String ugName = sc.nextLine();
System.out.print("Enter UG Student Marks (out of 100): ");
int ugMarks = sc.nextInt();
sc.nextLine(); // consume newline

// Input for PG student
System.out.print("Enter PG Student Name: ");
String pgName = sc.nextLine();
System.out.print("Enter PG Student GPA (0-10): ");
double pgGpa = sc.nextDouble();

// Create objects
Student s1 = new UGStudent(ugName, ugMarks);
Student s2 = new PGStudent(pgName, pgGpa);

System.out.println("\n--- Student Results ---");
s1.displayResult();
System.out.println();
s2.displayResult();

sc.close();
}
}

```

OUTPUT:

Sample Input:

Enter UG Student Name: Ram

Enter UG Student Marks (out of 100): 85

Enter PG Student Name: Ravi

Enter PG Student GPA (0-10): 9.2

Output:

--- Student Results ---

UG Student: Ram

Marks: 85

Percentage: 85.0%

PG Student: Ravi

GPA: 9.2 / 10

```
2 import java.util.Scanner;
3 class Student {
4     String name;
5
6     void displayResult() {
7         System.out.println("Student Result");
8     }
9 }
10 class UGStudent extends Student {
11     int marks;
12
13     UGStudent(String n, int m) {
14         name = n;
15         marks = m;
16     }
17     void displayResult() {
18         double percentage = (marks / 100.0) * 100;
19         System.out.println("UG Student: " + name);
20         System.out.println("Marks: " + marks);
21         System.out.println("Percentage: " + percentage + "%");
22     }
23 }
24 class PGStudent extends Student {
25     double gpa;
26     PGStudent(String n, double g) {
27         name = n;
28         gpa = g;
29     }
30     void displayResult() {
31         System.out.println("PG Student: " + name);
32         System.out.println("GPA: " + gpa + " / 10");
33     }
34 }
```

```

3     }
3 void displayResult() {
1     System.out.println("PG Student: " + name);
2     System.out.println("GPA: " + gpa + " / 10");
3 }
4 }
5
5 public class Main {
7 public static void main(String[] args) {
3     Scanner sc = new Scanner(System.in);
3     System.out.print("Enter UG Student Name: ");
3     String ugName = sc.nextLine();
1     System.out.print("Enter UG Student Marks (out of 100): ");
2     int ugMarks = sc.nextInt();
3     sc.nextLine();
4     System.out.print("Enter PG Student Name: ");
5     String pgName = sc.nextLine();
5     System.out.print("Enter PG Student GPA (0-10): ");
7     double pgGpa = sc.nextDouble();
3     Student s1 = new UGStudent(ugName, ugMarks);
3     Student s2 = new PGStudent(pgName, pgGpa);
3     System.out.println("\n--- Student Results ---");
1     s1.displayResult();
2     System.out.println();
2     s2.displayResult();
4     sc.close();
5 }
5 }
7

```

```

Enter UG Student Name: Ram
Enter UG Student Marks (out of 100): 85
Enter PG Student Name: Ravi
Enter PG Student GPA (0-10): 9.2
|
--- Student Results ---
UG Student: Ram
Marks: 85
Percentage: 85.0%

PG Student: Ravi
GPA: 9.2 / 10

```

2. Calculator Program (Method Overloading)

Description:

Create a Calculator class with multiple add() methods to calculate:

- Addition of 2 integers
- Addition of 3 integers
- Addition of 2 double numbers

Code:

```

import java.util.Scanner;

class Calculator {

    int add(int a, int b) {

        return a + b;
    }
}

```

```
}
```

```
int add(int a, int b, int c) {  
    return a + b + c;  
}
```

```
double add(double a, double b) {  
    return a + b;  
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        Calculator calc = new Calculator();  
  
        System.out.print("Enter two integers: ");  
        int x = sc.nextInt();  
        int y = sc.nextInt();  
        System.out.println("Sum of two integers: " + calc.add(x, y));  
  
        System.out.print("Enter three integers: ");  
        int p = sc.nextInt();  
        int q = sc.nextInt();  
        int r = sc.nextInt();  
        System.out.println("Sum of three integers: " + calc.add(p, q, r));  
  
        System.out.print("Enter two decimal numbers: ");  
        double a = sc.nextDouble();  
        double b = sc.nextDouble();  
        System.out.println("Sum of two doubles: " + calc.add(a, b));  
    }  
}
```

```

        sc.close();
    }
}

```

Output:

Sample Input:

Enter two integers: 10 20

Enter three integers: 5 10 15

Enter two decimal numbers: 2.5 3.5

Output:

Sum of two integers: 30

Sum of three integers: 30

Sum of two doubles: 6.0

```

2  import java.util.Scanner;
3  class Calculator {
4  @ int add(int a, int b) {
5      return a + b;
6  }
7  @ int add(int a, int b, int c) {
8      return a + b + c;
9  }
10 @ double add(double a, double b) {
11     return a + b;
12 }
13 }
14 public class Main {
15 @ public static void main(String[] args) {
16     Scanner sc = new Scanner(System.in);
17     Calculator calc = new Calculator();
18     System.out.print("Enter two integers: ");
19     int x = sc.nextInt();
20     int y = sc.nextInt();
21     System.out.println("Sum of two integers: " + calc.add(x, y));
22     System.out.print("Enter three integers: ");
23     int p = sc.nextInt();
24     int q = sc.nextInt();
25     int r = sc.nextInt();
26     System.out.println("Sum of three integers: " + calc.add(p, q, r));
27     System.out.print("Enter two decimal numbers: ");
28     double a = sc.nextDouble();
29     double b = sc.nextDouble();
30     System.out.println("Sum of two doubles: " + calc.add(a, b));
31     sc.close();
32 }

```

```

Enter two integers: 10 20
Sum of two integers: 30
Enter three integers: 5 10 15
Sum of three integers: 30
Enter two decimal numbers: 2.5 3.5
Sum of two doubles: 6.0

```

POST LAB EXERCISE

- ✓ Is return type important in method overloading and method overriding?
 - In method overloading, the return type alone is not considered for distinguishing methods.
 - Overloaded methods must differ in parameter list (number/type/order), not just return type.
 - In method overriding, the return type must be same or covariant (subtype of parent's return type).
 - If return type rules are violated in overriding, it leads to a compile-time error.

- ✓ Can you overload a method by changing only the return type?
 - No, a method cannot be overloaded by changing only the return type.
 - The compiler identifies overloaded methods based on the method signature, which includes method name and parameter list.
 - If only the return type is changed, it causes ambiguity for the compiler.
 - Therefore, parameters must be different to achieve valid method overloading.

- ✓ Can static methods be overridden? Can they be overloaded?
 - Static methods cannot be overridden because they belong to the class, not to the object.
 - Defining a static method with the same signature in a subclass is called method hiding, not overriding.
 - Static methods can be overloaded by changing the parameter list in the same class.
 - Overloading of static methods works the same way as instance method overloading.

- ✓ Can a method be overridden if the parameter list is different?
 - No, a method cannot be overridden if the parameter list is different.
 - For overriding, the method signature (name + parameters) must be exactly the same as in the parent class.
 - If the parameter list is different, it becomes method overloading, not overriding.
 - This rule ensures correct run-time polymorphism behavior in Java.

Result:

Thus the method overloading and overriding concepts were implemented and executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		